MoSE: Lower Bounds of Expert Granularity in Sparse Transformers on Resource-Constrained Hardware

Maaz Waheed

Independent Researcher

Email: mday2492@gmail.com

Abstract

Mixture-of-Experts (MoE) architectures increase model capacity through conditional computation, but conventional expert configurations can impose substantial memory and computational requirements on resource-constrained hardware. This paper investigates Mixture of Smaller Experts (MoSE), a sparse causal Transformer designed to explore practical expert granularity under a restricted GPU memory budget. MoSE uses four Transformer blocks, 16 routed micro-experts per layer, Top-2 token routing, and an unrouted persistent shared expert. The model contains 5,938,816 parameters, with 3,168,384 active parameters per forward pass. Training is evaluated on a mobile NVIDIA Quadro T1000 GPU with 4 GB GDDR6 memory using automatic mixed precision and memory-mapped token streams. The reported peak training VRAM is 860 MB, and sustained local throughput is 5,200–5,600 tokens/s. Across 3,400 optimization steps, validation perplexity decreases from an initial random baseline of approximately 9,934 to 77.38. Qualitative fill-in-the-middle samples show progression from incoherent fragments to structured Python expressions. Router observations indicate persistent token-structural specialization, including loop-associated behavior in Expert 13 and frame-exit behavior in Expert 6. These results establish an empirically viable micro-expert operating point; they do not constitute a universal theoretical lower bound.

Index Terms—*Mixture of Experts, sparse Transformers, conditional computation, expert granularity, edge training, memory-efficient language modeling, fill-in-the-middle.*

I. INTRODUCTION

Large language models achieve capacity through parameterized transformations, deep architectures, and extensive training. Dense Transformer feed-forward layers execute their principal transformations for every token, coupling model capacity to compute and memory costs. Mixture-of-Experts (MoE) architectures partially decouple these quantities by routing tokens to a selected subset of expert networks [1], [2].

Large-scale sparse systems such as Mixtral 8×7B demonstrate the utility of conditional computation, but coarse expert dimensions and total parameter storage remain challenging for many local or edge devices [3]. Fine-grained expert segmentation, as explored by DeepSeekMoE, provides a complementary strategy: divide expert capacity into smaller units and preserve shared computation for common patterns [4].

This work studies a compact operating point using micro-experts with intermediate dimension 128. The goal is to characterize feasibility and observed behavior on constrained hardware, not to claim a proven universal minimum. The title's "lower bounds" therefore refers to empirical granularity exploration; a formal lower bound on expert dimensions or achievable quality remains open.

Contributions

• A four-block sparse causal Transformer with 16 routed micro-experts per layer, Top-2 routing, and one persistent shared expert.

• An observed 860 MB peak VRAM profile on a 4 GB NVIDIA Quadro T1000, using FP16 AMP and memory-mapped token streams.

• A convergence record reaching reported validation perplexity 77.38 at Step 3400.

• Qualitative FIM progression and routing observations of persistent token-structural expert associations.

II. RELATED WORK

A. Sparse Mixture of Experts

Shazeer et al. introduced sparsely gated MoE layers, using learned gates to selectively activate a subset of feed-forward experts [1]. Fedus et al. proposed Switch Transformers, simplifying routing and developing load-balancing objectives for large sparse models [2]. MoSE follows conditional computation while reducing expert intermediate dimensions and targeting local GPU execution.

B. Fine-Grained Experts and Shared Computation

DeepSeekMoE studies fine-grained expert segmentation and shared-expert isolation to encourage specialization while retaining common computational pathways [4]. MoSE adopts these motivations at a smaller scale: 16 routed experts with dimension 128 and one unrouted shared expert in each block. The present experiment does not include an ablation that proves the shared expert is necessary.

C. Fill-in-the-Middle

Fill-in-the-Middle (FIM) trains autoregressive models to generate a missing span conditioned on surrounding prefix and suffix text [5]. We use a fixed Python scaffold as a qualitative probe. The samples are illustrative and do not establish code execution correctness.

III. THE MOSE ARCHITECTURE

A. Backbone and Micro-Expert Formulation

MoSE is a causal decoder-only Transformer with four blocks, model dimension d_model = 192, six attention heads, head dimension d_head = 32, and sequence length 256. Each block has N = 16 routed experts with intermediate dimension d_expert = 128 and one persistent shared expert with d_shared = 128.

$$E_i(x) = W_{i,2}\,\varphi(W_{i,1}\,x + b_{i,1}) + b_{i,2}$$
$$W_{i,1} \in \mathbb{R}^{128\times192},\quad W_{i,2} \in \mathbb{R}^{192\times128} \qquad (1)$$

Each expert maps a 192-dimensional token state through a 128-dimensional intermediate representation and back to 192 dimensions. Total parameters are 5,938,816; reported active parameters per forward pass are 3,168,384, and the reported sparsity ratio is 46.65%.

B. Top-2 Gating

$$z_t^{(l)} = W_g^{(l)} x_t^{(l)},\quad W_g^{(l)} \in \mathbb{R}^{16\times192}$$
$$p_{t,i}^{(l)} = \exp(z_{t,i}^{(l)}) \,/\, \Sigma_{j=1}^{16} \exp(z_{t,j}^{(l)}) \qquad (2)$$

The gate selects the two largest probabilities: $T_t^{(l)} = \text{Top-2}(p_t^{(l)})$. Selected weights are normalized over the chosen set, $g_{t,i} = p_{t,i} / \Sigma_{j\in T_t} p_{t,j}$, and the routed output is $R^{(l)}(x_t) = \Sigma_{i\in T_t^{(l)}} g_{t,i}^{(l)} E_i^{(l)}(x_t)$.

C. Persistent Shared Expert

$$S^{(l)}(x) = W_{s,2}^{(l)}\,\varphi(W_{s,1}^{(l)} x + b_{s,1}^{(l)}) + b_{s,2}^{(l)}$$
$$F^{(l)}(x_t) = R^{(l)}(x_t) + S^{(l)}(x_t) \qquad (3)$$

The shared expert is active for every token independent of router decisions. It is motivated as a common computational anchor alongside specialized routed paths. This semantic-anchor interpretation is a design rationale, not a causal conclusion; shared-expert ablation is required to test necessity.

D. Switch-Style Load Balancing

$$f_i = (1/T)\,\Sigma_t\ \mathbb{1}[i \in T_t],\quad P_i = (1/T)\,\Sigma_t\ p_{t,i}$$
$$L_aux = \alpha\, N\, \Sigma_i f_i P_i,\quad \alpha = 0.01,\quad N = 16 \qquad (4)$$
$$L = L_task + L_aux$$

The reported auxiliary loss stabilizes near 1.02–1.04. The objective encourages global load balance but does not require identical token-conditioned specialization.

IV. SYSTEM AND HARDWARE OPTIMIZATION

A. Hardware and Precision

The target is a mobile NVIDIA Quadro T1000 Laptop GPU (Turing architecture, 4.00 GB GDDR6 VRAM). Training uses FP16 automatic mixed precision through torch.amp. The reported peak training VRAM is 860 MB, approximately 21.5% of nominal 4 GB capacity.

B. Memory-Mapped Data Engine

Training and validation tokens are stored in flat continuous one-dimensional numpy.uint16 binary streams and accessed with np.memmap. train.bin contains 28,177,334 tokens (53.7 MB); val.bin contains 1,584,516 tokens (3.0 MB). The mapping avoids

materializing the full corpus as a conventional host array and accesses file-backed pages. This is not literal zero physical RAM usage: operating-system page cache, I/O, and host-to-device transfers remain.

C. VRAM Profile

Table II lists the reported allocation categories. The residual is calculated as the difference between the 860 MB peak and the specified weights, optimizer-state, and activation allocations; it is not independently measured.

TABLE I

SYSTEM AND PARAMETER PROFILE

Category	Specification / Reported Value
Transformer	4 layers; d_model=192; 6 heads; d_head=32; seq=256
MoSE	16 routed experts/layer; d_expert=128; 1 shared; Top-2
Parameters	5,938,816 total; 3,168,384 active; sparsity 46.65%
Hardware	NVIDIA Quadro T1000; 4.00 GB GDDR6; Turing
Peak VRAM	860 MB; FP16 AMP via torch.amp
Data engine	np.memmap; flat numpy.uint16 streams
Dataset	train: 28.2M tokens; val: 1.58M tokens
Throughput	5,200–5,600 tokens/s sustained locally

TABLE II

VRAM ALLOCATION PROFILE

Allocation	Amount
Weights	11.9 MB
Optimizer states	23.8 MB
Activations	~550 MB
Other runtime / residual	~274.3 MB (derived)
Total peak	860 MB
GPU capacity	4.00 GB
Peak capacity share	~21.5%

V. EMPIRICAL EVALUATION

A. Quantitative Convergence

The initial task loss is 9.2038, close to the uniform-vocabulary baseline $\ln(10{,}000) \approx 9.2103$. The initial perplexity is reported as approximately 9,934. Validation perplexity reaches 77.38 at Step 3400. Learning-rate values were not supplied and are marked NR rather than inferred.

The reported task and validation losses coincide at Steps 600, 1000, and 3400. These equalities may reflect rounding or measurement conventions; the supplied record does not establish the exact evaluation aggregation.

TABLE III

QUANTITATIVE CONVERGENCE

Step	Task Loss	Aux Loss	Val Loss	Perplexity
0	9.2038	1.1058	NR	~9,934
200	6.4856	NR	6.5385	691.24
400	5.7762	NR	5.8612	351.16
600	5.5332	NR	5.5332	253.00
1000	5.1680	NR	5.1680	175.56
3400	4.3488	NR	4.3488	77.38

B. FIM Infilling Evolution

The fixed probe is: <PRE>def add(a, b): [missing span] return result <SUF> [suffix] <MID>. The recorded evolution is shown in Table IV. Samples are qualitative and not necessarily executable.

TABLE IV

FIM INFILLING EVOLUTION

Step	Qualitative observed output / behavior
9	Incoherent random characters: “ender -------AAAUTSER...”
200	Punctuation, indentation, syntax primitives
400	Valid import forms: “from . import”; “from .. import”
600	Invocation and indentation patterns
1000	Syntactic clauses/signatures: “for in(,):”; “def_(,,): return”
3400	Cohesive structured expressions and assignment blocks

C. Routing Specialization and Entropy

Normalized entropy is $H_{norm}(p) = -\Sigma_i\ p_i \log(p_i)/\log(N)$. The reported normalized entropy falls from 75.0% to 45.3% for return and 47.6% for for. E13 retains dominant association with the reported “###” pattern on for loops at Steps 200, 400, and 1000. return and try are reported to converge on E6, interpreted as frame-exit/control-flow-related specialization. E14 is mentioned in the broader analysis, but no token-frequency matrix or precise assignment data are supplied; no separate quantitative E14 conclusion is drawn.

The auxiliary loss range and entropy observations are compatible with stable specialization alongside load balancing. Stronger evidence would require full routing matrices, token-conditioned frequencies, independent runs, and controlled ablations.

TABLE V

ROUTING SPECIALIZATION MATRIX

Token / pattern	Reported routing behavior	Entropy / persistence
for loops	Expert 13 (E13) dominant	No drift across checkpoints
return	Converges to Expert 6 (E6)	75.0% → 45.3%
try	Converges to Expert 6 (E6)	Frame-exit association
for token	Token-conditioned concentration	Normalized entropy 47.6%
Auxiliary loss	α=0.01; Switch-style	Stabilized ~1.02–1.04

VI. DISCUSSION, LIMITATIONS, AND FUTURE WORK

A. Expressivity and Routing Overhead

The number of unordered Top-2 expert pairs among 16 experts is C(16,2) = 120 per layer. Across four layers, $120^4 = 207{,}360{,}000$ possible sequences of pair choices. This is a combinatorial count of routing configurations, not distinct learned functions. Routing, dispatch, aggregation, and memory-access overhead may become proportionally more significant as experts shrink; hence smaller experts do not guarantee higher throughput.

B. Limitations

• The model is 5.94 M parameters; conclusions do not directly scale to billion-parameter systems.

• Only one GPU platform and a limited set of checkpoints are reported.

• Learning rate, optimizer specifics, batch size, scheduler, and some measurement conventions are absent from the supplied record.

• FIM evidence is qualitative and based on a fixed Python prompt; it does not establish functional code correctness.

• Routing specialization lacks complete token-to-expert frequency matrices, statistical tests, and independent-run reproducibility.

• No expert-size, Top-k, shared-expert, or auxiliary-loss ablation is provided; 128 is an empirical operating point, not a proven theoretical minimum.

C. Dynamic NVMe-to-VRAM Streaming

A future system could keep expert weights on NVMe and cache selected experts in VRAM using a least-recently-used (LRU) policy. Such a system would require asynchronous transfer, residency tracking, prefetching, cache eviction, synchronization, and—during training—management of gradients and optimizer states. The trade-off depends on expert access frequency, storage bandwidth, transfer latency, and compute time.

VII. CONCLUSION

MoSE explores a fine-grained sparse Transformer operating point for local resource-constrained training. A four-block model with 16 routed micro-experts per layer, Top-2 routing, and a persistent shared expert contains 5,938,816 parameters and reportedly trains with 860 MB peak VRAM on a 4 GB Quadro T1000. Validation perplexity falls to 77.38 by Step 3400, while qualitative FIM samples and routing observations suggest progressive syntax acquisition and token-structural expert associations. The evidence supports feasibility at d_expert = 128, but does not establish a universal lower bound. Controlled expert-dimension ablations, stronger code benchmarks, full routing statistics, and NVMe expert streaming remain important next steps.

REFERENCES

[1] N. Shazeer et al., “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer,” in Proc. ICLR, 2017.

[2] W. Fedus, B. Zoph, and N. Shazeer, “Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity,” J. Mach. Learn. Res., vol. 23, no. 120, pp. 1–39, 2022.

[3] A. Jiang et al., “Mixtral of Experts,” arXiv:2401.04088, 2024.

[4] DeepSeek-AI, “DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models,” arXiv:2401.06066, 2024.

[5] M. Bavarian et al., “Efficient Training of Language Models to Fill in the Middle,” arXiv:2207.14255, 2022.

[6] A. Vaswani et al., “Attention Is All You Need,” in Adv. Neural Inf. Process. Syst., vol. 30, 2017.

[7] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale,” in Adv. Neural Inf. Process. Syst., vol. 35, 2022.

[8] J. Hoffmann et al., “Training Compute-Optimal Large Language Models,” in Adv. Neural Inf. Process. Syst., vol. 35, 2022.